GoMaa

Raga Vidya v3

Carnatic R■ga Intelligence Engine

Architecture · Solution Document · User Guide

7,599	35+11	926	4-Pass
Ragas in DB	Tala Types	Inline Ragas	Tala Detection

Version 3.0 · June 2025 · Node.js · Pure JS · Offline-first

Built on melakartajanyaragalist.csv · 7,599 ragas from Carnatic tradition

Table of Contents

1 System Overview

1.1 Purpose & Goals

GoMaa Raga Vidya v3 is an offline-first, open-source Carnatic music intelligence system. It identifies ragas from audio, generates sheet music (MusicXML), detects tala structure, evaluates swaras, sahityam and gamakas, and allows composing new pieces in any of 7,599 ragas from the Carnatic tradition — all without requiring an internet connection or cloud subscription.

Design principles:

- **Accuracy over speed** — scale-exact F1 matching against 7,599 ragas, not just 72 melakartas.
- **Musicologically correct** — tala detected from rhythmic cycle structure, not BPM guessing.
- **Offline-first** — entire detection engine runs in-browser via Web Audio API when no server is available.
- **No native binaries** — pure JavaScript/Node.js; no Python, no FFmpeg, no native audio libs required.
- **Complete data** — all 7,599 rows from melakartajanyaragalist.csv are unique (MelakartaJanyaRef is the primary key); no deduplication.

7,599	926	35	11	a→j
Ragas in Server DB	Ragas in Browser (inline)	Carnatic Talas	Hindustani Talas	Audio Pipeline Steps

1.2 Technology Stack

Runtime	Node.js ≥ 16 (server) + Browser Web Audio API (offline)
Framework	Express 4.x, Multer, CORS
Database	sql.js (SQLite in-memory, no native binary)
Raga DB	models/ragas_db.json — 7,599 entries from melakartajanyaragalist.csv
Sheet Music	MusicXML 3.1 generated server-side; rendered by OpenSheetMusicDisplay v1.8
MIDI	Binary MIDI generated in pure JS, base64-encoded for download
Audio (server)	Raw byte autocorrelation — no FFT library required
Audio (browser)	Web Audio API AudioBuffer → autocorrelation pitch extraction
Frontend	Vanilla HTML/CSS/JS, single-file SPA (index.html, 457 KB)
Fonts	Cormorant Garamond + DM Mono + Lora (Google Fonts CDN)

1.3 Key Improvements in v3

Issue	v2 Behaviour	v3 Fix
Raga DB size	72 melakartas only	**7,599 ragas (all CSV rows)**

Raga matching	Cosine on 12-bin chroma only	**Scale-exact F1 + cosine (70/30)**
Bilahari detection	Returned Vanaspati (#4)	**Exact match: ref=3540, mela=28**
Ekadantam detection	Returned Vanaspati (#4)	**Exact match: ref=1716, mela=16**
Tala detection	BPM → hardcoded name (wrong)	**4-pass autocorrelation on 3 segments**
Tala framework	Only 4 names	**35 Carnatic (7×5) + 11 Hindustani**
Audio aroha/avaroha	Taken from KB only	**Detected from ascending/descending pitch**
Gamaka detection	Not detected	**Kampita/andola/spurita/neravel from vibrato index**
Instrument detection	Not present	**Spectral band energy + ZCR classifier**
Tala in UI	Not shown	**r-tala + r-tempo cells with detected values**
Detected aroha badge	Not shown	**Shows audio-detected vs KB aroha when different**

2 Architecture

2.1 Full System Architecture

GoMaa Raga Vidya v3 is a Node.js monorepo with a clear separation between browser-side (offline) and server-side (enhanced) execution paths. Both paths share the same detection logic and produce identical output structures.

BROWSER (Offline Mode — file:// or no server)	
Audio File / Mic / YouTube	→ Web Audio API → Float32Array
Float32Array	→ Autocorrelation pitch extraction
Pitch frames	→ _detectScaleBrowser() → semitone set
Semitone set	→ _scoreAllV3() → F1 + cosine → raga match
Raga match	→ _detectTalaBrowser() → 4-pass ACF → tala
Tala result	→ _detectGamakasBrowser() → kampita/andola
All results	→ buildMusicXML() → OpenSheetMusicDisplay render
SERVER (Enhanced Mode — npm start)	
POST /api/recognize/buffer	→ Raw audio bytes (no FormData CORS issue)
extractPitchFrames()	→ YIN autocorrelation per 512-sample hop
detectAudioScale()	→ 12-bin chroma energy, 20% threshold
detectArohaAvaroha()	→ Ascending/descending trajectory split
detectRaga() via ragaModel.js	→ ragas_db.json 7,599 ragas F1+cosine
detectGamakas()	→ Vibrato index, crossing rate, neravel
detectInstruments()	→ Band energy + ZCR spectral classifier
detectTala()	→ 3-pass ACF on segments → ALL_TALAS vote
buildSahityamGrid()	→ Beat-aligned swara tokens with |/|| markers
generateSheetMusicXml()	→ MusicXML 3.1 + MIDI base64
JSON response	→ {raga, aroha, avaroha, talaResult, audioAnalysis, lyricsData, sheetMusicXml, midiData, stems}

2.2 Server-Side Components

backend/server.js	Express entry point. Mounts all routes. Serves SPA from apps/web/
backend/routes/recognize.js	Main audio analysis route. Implements the full a→j pipeline (1,247 lines).
backend/routes/compose.js	Generates MusicXML + MIDI for a selected raga with user lyrics.
backend/routes/search.js	Hybrid text + vector search. Autocomplete suggestions.
backend/routes/dataset.js	File upload + folder scan + auto-ingest.
backend/routes/ingest.js	Triggers ingestDataset.js from the UI dashboard.

core/ai/ragaModel.js	Raga detection engine. Loads ragas_db.json. Scale-exact + cosine matching.
core/ai/sheetMusicEngine.js	MusicXML 3.1 and MIDI generation from swara sequences.
core/ai/audioEmbedding.js	64-dim embedding vector for similarity search.
core/ai/fusionEngine.js	Combines fingerprint + embedding + raga scores.
core/db/sqlite.js	sql.js wrapper. Auto-creates schema. No rollback errors.
core/vector/annIndex.js	In-memory ANN index for vector similarity search.
models/ragas_db.json	7,599 ragas from CSV. Primary key: MelakartaJanyaRef. 1.2 MB.
models/knowledge_base.json	72 Melakarta ragas with mood/gamaka/chakra metadata. Fallback.

2.3 Browser-Side Components (index.html)

_V3_RAGA_DB	926 ragas inline: all melakartas + popular janyas. Loaded at page open.
_V3_RAGA_NORMS	Pre-normalised chroma vectors for cosine similarity at boot time.
_scoreAllV3()	Scale-exact F1 (70%) + cosine (30%) scoring against all 926 inline ragas.
detectRagaInBrowser()	Full pipeline: pitch → scale → raga → tala → gamakas → lyricsData.
_detectTalaBrowser()	4-pass ACF tala detection: beat period, cycle period, onset peaks, segments vote.
_TALA_DB_BROWSER	35 Carnatic + 11 Hindustani tala definitions with anga groups and tradition.
_detectGamakasBrowser()	Kampita, andola, spurita detection from frequency deviation + crossing rate.
_extractAudioLyricsData()	Tala-aware beat grouping → swara strings with | and || cycle markers.
buildMusicXML()	Inline MusicXML builder. Works fully offline. Composition-first layout.
renderScoreSVG()	Inline SVG staff notation renderer with Carnatic underline beam notation.
renderOSMD()	OpenSheetMusicDisplay renderer for professional notation from MusicXML.

3 Raga Detection Engine

3.1 Detection Hierarchy

Three tiers, evaluated in priority order. The first successful match returns immediately.

Priority	Method	Confidence	When Used
1 — Highest	Filename token match	92% (fixed)	File named "bilahari_concert.mp3"
2 — Audio	Scale-exact F1 + cosine (70% scale, 30% cosine)	Computed	Audio bytes available (>512 bytes)
3 — Fallback	Hash-based deterministic (score × 0.65 penalty)	Low	No audio, no filename hint

3.2 Scale-Exact Matching Algorithm

The core innovation in v3. Instead of matching only a 12-bin chroma vector (which confuses ragas sharing most notes), v3 uses the **aroha semitone fingerprint** of every raga from the CSV. The F1-like score measures how precisely the detected audio scale matches each raga's ascending scale (aroha).

```
// F1-like score: 2 × |detected ∩ raga_aroha| / (|detected| + |raga_aroha|) arohaF1 = 2 * arohaIntersection / (detectedSemis.size + arohaSet.size) allF1 = 2 * allIntersection / (detectedSemis.size + allRagaSet.size) scaleScore = 0.6 * arohaF1 + 0.4 * allF1 combined = 0.30 * cosineScore + 0.70 * scaleScore
```

3.3 Raga Database — 7,599 Entries

The server loads **models/ragas_db.json** containing all 7,599 rows from melakartajanyaragalist.csv. Each row is unique by its **MelakartaJanyaRef** primary key — no deduplication is performed. The same raga name may appear multiple times under different parent melakartas or with different aroha patterns, and all variants are retained.

Primary key	MelakartaJanyaRef (integer, unique per row in CSV)
Fields stored	ref, n (name), m (melakarta), j (janya), a (aroha), v (avaroha), c (chroma[12]), as (aroha semis), vs (avaroha semis)
Total entries	7,599 ragas (all CSV rows with valid aroha/avaroha)
Browser inline	926 ragas (all 72 melakartas + 854 popular janyas)
Semitone vectors	Pre-computed sorted arrays of unique semitones in aroha and avaroha

3.4 Discrimination Proof: Bilahari vs Vanaspati vs Ekadantam

In v2, uploading "Ekadantam Bhajeham" was returned as **Vanaspati** because only the 72 melakartas were in the DB — the janya ragas bilahari and shrIEkadantA were absent. The cosine similarity then incorrectly matched to the nearest melakarta.

Raga	DB Ref	Mela	Aroha	Aroha Semitones	Note Count	v3 Result
bilahari	3540	28	S R2 G3 P D2 S	[0, 2, 4, 7, 9]	5 (audava)	✓ Correct
shrIEkadantA	1716	16	S R1 G3 P D2 S	[0, 1, 4, 7, 9]	5 (audava)	✓ Correct

vanaspati	170	4	S R1 G1 M1 P D2 N2 S	[0, 1, 2, 5, 7, 9, 11]	7 (sampoorna)	Never matched to 5-note audio

Key: bilahari has no Ma (semi 5/6) and no Ni (semi 10/11). Vanaspati has 7 notes including M1 (5) and N2 (11). The F1 score of a 5-note audio against a 7-note raga is at most 0.714, while against bilahari's exact 5 notes it is 1.000.

4 Audio Analysis Pipeline (a → j)

4.a Pitch Extraction

extractPitchFrames()

YIN-inspired autocorrelation per 512-sample hop window. Range: 80–1200 Hz. Returns {freq, midi, semi, confidence, rms} per frame. Silence below RMS 0.005 is marked as silent and excluded from scale detection.

4.b Scale Detection

detectAudioScale()

Accumulates energy per semitone (0–11) from all voiced frames weighted by confidence. Adaptive 20% threshold retains the right number of pitch classes: 5 for audava (pentatonic) ragas like bilahari, 7 for sampoorna ragas.

4.c Aroha / Avaroha Detection

detectArohaAvaroha()

Segments pitch frames into ascending (aroha) and descending (avaroha) runs by counting up/down movements in a 20-frame window. Maps frames to swaras via semiToSwara built from the matched raga's scale.

4.d Raga Identification

detectRaga() via ragaModel.js

Loads ragas_db.json (7,599 entries). Combined score = 30% cosine + 70% scale-exact. If audio-detected scale is available and raga was not filename-matched, detectRagaFromScale() is called for a second refinement pass.

4.e Swara Evaluation

evaluateSwaras()

Maps each voiced pitch frame to the nearest Carnatic swara using semiToSwara. Tags each frame as attack, sustain, or silence. Returns per-frame {time, swara, freq, midi, gamaka, confidence}.

4.f Sahityam Grid

buildSahityamGrid()

Groups frames into tala beats using the detected BPM. The dominant swara per beat becomes a token. Tokens are assembled with | (anga boundary) and || (cycle end) markers. Output: swaras_pallavi, swaras_anupallavi, swaras_charanam strings.

4.g Lyrics Evaluation

buildLyricsData()

Merges audio-detected swara strings (from f) with KB text lyrics (pallavi, anupallavi, charanam in Telugu/Sanskrit) when the raga composition is in the demo KB. Audio swaras take precedence for the score; KB text fills the lyric lines.

4.h Instrument Detection

detectInstruments()

Divides audio into 4 spectral bands (low/mid-low/mid-high/high). Computes zero-crossing rate (ZCR). Rules: vocal = strong midrange + moderate ZCR; veena = low+mid-low dominant; flute = high content; mridangam = peaky low + high ZCR; tampura = low dominant + very low ZCR (sustained drone).

4.i Gamaka Detection

detectGamakas()

Measures pitch deviation (std/mean) and zero-crossing rate of the frequency envelope in 30-frame windows. kampita: dev>0.008 and rate>4 Hz (fast vibrato). andola: dev>0.015 and rate 1–4 Hz (slow oscillation). spurita: dev 0.004–0.008 (light grace). neravel: detects repeated swara sequences at different pitch levels.

4.j Sheet Music & Score

generateSheetMusicXml() + generateMidi()

Produces MusicXML 3.1 with: pallavi swaras + sahityamu aligned per note, anupallavi, charanam, and scale reference at the bottom (Patantara.com layout). MIDI encodes veena melody + tampura drone + mridangam pattern on 4 tracks. Sheet is rendered in-browser by OpenSheetMusicDisplay and the custom SVG renderer.

5 Tala Detection System

Tala is the rhythmic cycle structure of Indian classical music. GoMaa v3 detects tala from the **cycle length in beats** — not from BPM alone. A Rupakam (6 beats) and a Misra Chapu (7 beats) can have the same tempo; only the cycle structure distinguishes them.

5.1 Carnatic Tala Framework — 35 Talas

Carnatic music uses 7 core talas × 5 jatis = 35 standard talas. Each jati defines the length (beat count) of the laghu anga. The other angas (dhrutam = 2, anudhrutam = 1) are fixed. Clap falls on the first beat of every anga (unlike some Hindustani talas).

Tala	Anga Sequence	Chatusra (4)	Tisra (3)	Khanda (5)	Misra (7)	Sankirna (9)
Eka	[L]	4 beats	3	5	7	9
Rupaka	[D+L]	6 ✓ Adi-like	5	7	9	11
Triputa	[L+D+D]	**8 (Adi)**	7	9	11	13
Jhampa	[L+U+D]	7	6	8	10	12
Matya	[L+D+L]	10	8	12	16	20
Ata	[L+L+D+D]	12	10	14	18	22
Dhruva	[L+D+L+L]	14	11	17	23	29

★ *Adi tala = Chatusra Jati Triputa = 4+2+2 = 8 beats (most common in Carnatic concerts)*

5.2 Hindustani Tala Framework — 11 Talas

Tala	Beats	Sections	Clap Pattern	Notes
Tintal / Teental	16	4+4+4+4	X 2 0 3	Most common. Khali on beat 9.
Ektal	12	2+2+2+2+2+2	X 0 2 0 3 4	Slow khayal. Khali on beats 3,5.
Jhaptal	10	2+3+2+3	X 2 0 3	Medium tempo.
Rupak	7	3+2+2	0 2 3	Starts with khali — unusual.
Dhamar	14	5+2+3+4	X 2 0 3	Used for Holi songs, Dhrupad.
Keherwa	8	4+4	X 0	Light classical, thumri, bhajan.
Dadra	6	3+3	X 0	Light forms, folk-based.
Deepchandi	14	3+4+3+4	X 2 0 3	Thumri, semi-classical.
Tilwada	16	4+4+4+4	X 2 0 3	Slow khayal, similar to Tintal.
Tivra	7	3+2+2	X 2 3	Starts with clap (unlike Rupak).
Chautal	12	4+4+2+2	X 0 2 0 3 4	Dhrupad style.

5.3 Multi-Pass Cycle Detection Algorithm

The algorithm detects the tala cycle from the audio's rhythmic structure using autocorrelation of the onset strength envelope — not from BPM alone.

Pass 1 — Beat period

Autocorrelation of onset energy in the 200–3000 ms range. Finds the tactus (beat period T in frames).

Pass 2 — Cycle period

Long-range ACF in the 800–25000 ms range. Finds the tala cycle period C. Beats-per-cycle = round(C / T).

Pass 3 — Onset-based BPM

Finds onset peaks, computes inter-onset intervals, takes median → second BPM estimate.

Pass 4 — Cross-segment vote

3 independent segments of the audio each run their own beat-period ACF. 5 BPM estimates voted with round-to-nearest-5 bucketing. Median cycle length taken from 3-pass cycle votes.

Tala match

Detected beat count compared against ALL_TALAS (35 Carnatic + 11 Hindustani). Candidates within ±2 beats scored by priority. Winner = highest-priority match. Adi (priority=15) wins ties.

6 API Reference

6.1 Recognition Endpoints

Method	Endpoint	Input	Description
POST	/api/recognize	multipart/form-data field: audio	Upload audio file. Full a→j analysis.
POST	/api/recognize/buffer	Raw bytes. Header: x-filename	ArrayBuffer upload (avoids FormData CORS). Preferred from browser
POST	/api/recognize/url	JSON: {url}	Fetch remote .mp3/.wav URL and analyse. YouTube blocked.
POST	/api/recognize/path	JSON: {filePath}	Analyse file at server path.
GET	/api/recognize/stems/:id	—	Get instrument stem breakdown for saved recording.
GET	/api/recognize/lyrics/:id	—	Get lyrics/sahityam data for saved recording.

Recognition Response Fields

raga	Detected raga name (from ragas_db.json)
ragaNumber	Parent melakarta number (1–72)
aroha	Raga's ascending scale from DB
avaroha	Raga's descending scale from DB
detectedAroha	Aroha detected from actual pitch trajectory in the audio
detectedAvaroha	Avaroha detected from actual pitch trajectory in the audio
confidence	high / medium / low
detectionScore	Combined F1+cosine score (0–1)
detectionSource	filename / audio-scale / hash
audioAnalysis	{detectedScale, chroma, estimatedTempo, talaResult, detectedGamakas, instruments}
talaResult	{tala, beats, sections, tradition, bpm, confidence, candidates}
gamakas	Detected ornaments: kampita, andola, spurita, neravel
instruments	[{name, label, confidence}] ordered by confidence
lyricsData	{swaras_pallavi, swaras_anupallavi, swaras_charanam, pallavi, anupallavi, charanam, tala, tempo}
sheetMusicXml	Complete MusicXML 3.1 document string
midiData	Base64-encoded MIDI file (4 tracks: veena, tampura, mridangam, violin)
topCandidates	Top 5 matching ragas with scores and aroha
isRagamalika	Boolean: multiple ragas detected in the audio
ragamalikaSegments	Array of {start, end, raga, aroha, avaroha, score}

6.2 Other Endpoints

GET	/api/search	q=&raga=&mood=&limit=	Hybrid text + vector search
GET	/api/search/suggest	q=	Autocomplete: ragas, titles, artists from KB
GET	/api/search/ragas	—	All distinct ragas in the library
POST	/api/compose	JSON: {title,raga,tala,tempo,instrument,sections}	Generate MusicXML + MIDI
GET	/api/compose	—	List saved compositions
GET	/api/compose/:id	—	Fetch one composition
GET	/api/sheet/:id	—	Download MusicXML file
GET	/api/midi/:id	—	Download MIDI file
GET	/api/dataset/status	—	Song/raga/composition counts
POST	/api/dataset/upload	multipart files	Upload MP3/WAV/JSON/XML
POST	/api/dataset/scan	JSON: {folder?}	Scan server folder and ingest
GET	/api/ragas	—	Full 72-raga knowledge base JSON
GET	/api/health	—	Server status

7 User Guide

7.1 Installation & Quick Start

Step 1. Unzip

```
unzip GoMaa-RagaVidya-v3-final.zip && cd gomaa_v3
```

Extract the downloaded zip

Step 2. Install

```
npm install
```

Install Express, Multer, sql.js (~30 seconds)

Step 3. Ingest

```
npm run ingest
```

Creates demo library from 72-raga KB (or scans datasets/ for MP3s)

Step 4. Start

```
npm start
```

Server starts on http://localhost:3000

Offline (no server) mode:

Open **apps/web/index.html** directly in any browser. The app detects file:// protocol and switches to 100% in-browser mode. Drag-and-drop MP3/WAV files work immediately. URL fetching and dataset scanning require the server.

7.2 Recognize Audio — File Upload

- Click "■ Recognize" in the top navigation bar.
- The File Upload tab is selected by default.
- Drag and drop any MP3, WAV, FLAC, OGG, or M4A file onto the drop zone — or click to browse.
- The file name, size, and an Analyze button appear below the drop zone.
- Click ■ **Analyze**. A progress bar shows while analysis runs.
- Results appear automatically below. Scroll down to see the full output.

Tip: Name your file with the raga name for highest accuracy: "bilahari_concert.mp3"

7.3 Live Microphone Recording

- Click the ■ **Live Record** tab.
- Click ■ **Start Recording**. The browser will ask for microphone permission — click Allow.
- Sing, play an instrument, or hold your device near a speaker.
- The animated bars show the microphone is active. A timer counts up.
- Click ■ **Stop & Analyze**. The recording is processed immediately.
- Click ■ **Play Recording** to hear your recording back while viewing results.
- For best results: record at least 15–20 seconds of melodic content.

7.4 YouTube / URL Analysis

Direct URL (MP3/WAV):

- Click the ■ **URL / Stream** tab, then ■ **Direct Audio URL**.
- Paste a direct link to a .mp3 or .wav file (e.g., from Zenodo or Archive.org).
- Click ■ **Analyze**. The server fetches and analyses the file.
- YouTube, Spotify, and SoundCloud URLs cannot be fetched directly (they block server requests).

YouTube (embed + mic capture):

- Click ■ **YouTube Player** sub-tab.
- Paste a YouTube URL and click ■ **Load**.
- Click ■ **Record & Analyze** — this starts the microphone.
- Press Play on the YouTube embed. Your mic captures the audio playing in the room.
- Click Stop when done. The captured mic audio is analysed.

7.5 Reading the Results Screen

Raga name (large)	The identified raga from the 7,599-entry database.
Melakarta # / Chakra	Parent melakarta number and chakra group (Indu/Netra/Agni…).
■roha■am ↑	Ascending scale from the database for the identified raga.
Avaroha■am ↓	Descending scale from the database.
Audio-detected badge	If the pitch trajectory produced a different aroha from the DB aroha, a gold badge shows the detected version.
Confidence	high (>75% score), medium (>50%), or low.
Score %	Combined F1+cosine score expressed as a percentage.
Source	filename = matched by file name. audio-scale = from pitch analysis. hash = fallback.
T■la	Detected tala with beat count. E.g. "Adi (8 beats)".
Tempo	Estimated BPM from 3-pass onset autocorrelation.
Gamakas	Detected ornaments: kampita, andola, spurita, neravel.
Instruments	Detected instruments by spectral analysis, ordered by confidence.
Top Candidates	Top 5 raga matches with their scores for review.
Ragamalika banner	Appears if multiple ragas are detected in one recording. Shows timeline.

7.6 Music Score & Sheet Music Tabs

After recognition, the ■ **Music Score · Swaramu · Sahityamu** card appears with four sub-tabs:

■ Score	OpenSheetMusicDisplay renders full MusicXML notation. Below it, the SVG inline renderer shows Carnatic-style staff with swara names and sahityamu syllables aligned per note. Underlines below swara names indicate speed: 1 line = 2/beat, 2 lines = 4/beat.

■ Swaramu	Swara boxes for each section (Pallavi, Anupallavi, Charanam, Alapana). Each box shows the Carnatic swara name (S R2 G3…) with its Western equivalent below. Hover for semitone information. V■di/Samv■di (Sa, Pa) are gold-bordered.
■ Sahityamu	The lyrics/sahityam panel. Shows Telugu/Sanskrit text with each syllable aligned to its swara in a table grid (one column per note, exactly like Patantara.com). Section labels: Pallavi, Anupallavi, Charanam.
■ MIDI / Stems	In-browser synthesizer with 6 instrument tracks. Volume sliders, Solo buttons, Tala beat visualizer (8 cells for Adi), Swara highlight during playback. Download MIDI (4 tracks) and MusicXML buttons.
■ MusicXML	Downloads a .musicxml file openable in MuseScore, Finale, Sibelius, or Patantara.
■ MIDI	Downloads a .mid file with 4 tracks: veena (ch0), tampura (ch1), mridangam (ch9), violin (ch3).
■ MuseScore	Opens musescore.org in a new tab. Import the downloaded MusicXML file there.
■ Songscription	Opens songscription.ai — AI audio-to-sheet transcription service.

7.7 Compose a New Piece

- Click ■ **Compose** in the navigation bar.
- Enter a **Composition Title**.
- Select a **R■ga** from the dropdown (all 72 melakartas + popular janyas).
- Choose a **T■la** (Adi, Rupakam, Misra Chapu, Khanda Chapu, Jhampai, Dhruva, Matya, Triputa).
- Set **Tempo** in BPM (40–240). 80 BPM is typical Adi. 60 BPM for vilambita (slow).
- Select **Language** (Telugu, Sanskrit, Tamil, Kannada, Malayalam, Hindi, English).
- Toggle **Instruments**: Veena, Violin, Flute, Sitar, Mridangam, Tabla, Ghatam, Harmonium…
- Enter lyrics in the **Pallavi**, **Anupallavi**, and **Charanam** text areas.
- Click ■ **Generate Composition**. MusicXML and MIDI are generated server-side.
- Download MusicXML or MIDI from the result panel. Open in MuseScore for full notation.

*Tip: Click ■ **Open in Compose** → on any recognition result to pre-fill the raga and detected swaras.*

7.8 Search Your Library

- Click ■ **Search** in the navigation bar.
- Type in the search box: raga name, song title, artist, or mood.
- Autocomplete suggestions appear: r■gas (from KB), song titles, artists.
- Filter by mood using the chips below the search bar.
- Click any result row to open the raga detail modal.
- In the modal, click ■ **Compose** → to open that raga in the Compose page.

7.9 Dataset Management

- Click ■■ **Dataset** in the navigation bar.
- The status panel shows Songs / Ragas / Compositions counts from the SQLite database.
- To add audio: drag MP3/WAV/FLAC files onto the ■ **Upload Files** zone. Each file is automatically raga-analysed and saved.
- To upload a JSON knowledge base: drag a .json file with a "ragas" array.
- To upload notation: drag .xml, .mxl, or .musicxml files.
- To scan a server folder: enter the full path (e.g., /home/user/music) and click ■ **Scan & Ingest**.
- References to free Carnatic datasets (Saraga/Zenodo) are listed at the bottom of the Dataset page.

7.10 Tips for Best Accuracy

Name your files	Include the raga name: "Kharaharapriya_alapana.mp3" → 92% confidence from filename match.
Audio quality	Clear recordings with minimal background noise give better pitch extraction. Lossless WAV > compressed MP3 for analysis.
Recording length	At least 15–20 seconds for tala detection. At least 30 seconds for ragamalika detection.
Melody clarity	Single-instrument or vocal-only recordings are more accurate than dense ensemble mixes.
Offline vs server	Server mode (npm start) performs YIN pitch extraction on raw bytes. Browser mode uses Web Audio API. Both are accurate for clear audio.
Janya ragas	All 7,599 janya ragas are in ragas_db.json. If a janya raga is not in the 926 inline DB, the server will still correctly identify it.
Audava vs sampoorna	Pentatonic ragas (5-note like bilahari, mohanam) are confidently separated from 7-note ragas by the adaptive threshold.
Microphone placement	Hold the device 20–30 cm from the speaker/instrument. Do not cup the microphone.

8 Troubleshooting & FAQ

Q: Why does the app say "offline mode"?

A: The server is not running. Start it with: npm start In offline mode all recognition still works via Web Audio API in the browser.

Q: Microphone button is greyed out.

A: Check browser permissions. In Chrome: Settings → Privacy → Microphone → allow localhost. In Safari: the browser must be accessed via http:// not file://.

Q: CORS error / FormData postMessage error.

A: This is a browser extension conflict (usually React DevTools). The app uses /api/recognize/buffer (raw ArrayBuffer) instead of FormData to avoid this. Try disabling browser extensions or use an incognito window.

Q: Wrong raga detected.

A: If the audio is a janya raga not in the 926 inline DB, run the server: npm start. The server has all 7,599 ragas. Also check: does the filename contain the raga name? That gives 92% confidence.

Q: "Vanaspati" returned for Bilahari/Ekadantam.

A: This was the v2 bug. v3 is fixed — bilahari (ref=3540) and shrIEkadantA (ref=1716) are both in the database with distinct aroha semitone sets.

Q: MusicXML does not open in MuseScore.

A: The generated XML is MusicXML 3.1 Partwise. Open MuseScore 4, File → Open, select the .musicxml file. Alternatively, drag it onto the MuseScore window.

Q: npm run ingest creates no entries.

A: No MP3 files were found in datasets/. The script creates demo entries from the 72-raga KB instead. Drop your MP3 files in datasets/audio/ and run npm run ingest again.

Q: The server crashes on startup.

A: Check Node.js version: node --version (must be ≥16). Run: npm install (may need to reinstall). Check that models/ragas_db.json exists (1.2 MB file in the zip).

Q: Tala shows "—" in the results.

A: Tala detection requires at least 8 beat-periods of audio (~5–10 seconds at 80 BPM). Record longer or upload a longer file.

Q: Can I use this with Hindustani music?

A: Yes. The tala detector includes 11 Hindustani talas. Raga detection works for ragas in the CSV — many Hindustani ragas appear as janyas of melakartas.

9 Glossary of Carnatic Music Terms

R■ga (■■■■)

The melodic framework of Indian classical music. Defined by its aroha (ascending scale), avaroha (descending scale), characteristic phrases, and emotional mood (bh■va).

Melakarta

One of 72 parent scales in Carnatic music from which all janya ragas are derived. Organised into 12 chakras of 6 each.

Janya R■ga

A raga derived from a melakarta by taking a subset of its notes or using vakra (zigzag) phrases. The CSV contains 7,599 such ragas.

■roha■am (■■■■■■)

The ascending scale of a raga: S R G M P D N S. May skip notes (audava) or zigzag (vakra).

Avaroha■am (■■■■■■■)

The descending scale: S N D P M G R S. May differ significantly from the aroha.

Svara / Swara

One of 12 pitch positions: S, R1/R2/R3, G1/G2/G3, M1/M2, P, D1/D2/D3, N1/N2/N3. Sa (S) and Pa (P) are fixed; others vary by raga.

Gamaka

Ornamental embellishment applied to a swara. Types in v3: kampita (vibrato), andola (slow oscillation), spurita (grace note), neravel (melodic echo).

Kampita (■■■■■)

Rapid oscillation around a note, like vibrato. Most common gamaka. Identified by frequency deviation >0.8% and crossing rate >4 Hz.

Andola (■■■■■)

Slow, wide oscillation between two notes. Like a pendulum swing. Crossing rate 1–4 Hz.

Neravel

A melodic phrase sung to one syllable at multiple pitch levels. Identified by repeated swara sequence patterns.

T■la (■■■■)

The rhythmic cycle. In Carnatic music: 7 core talas × 5 jatis = 35 standard talas. Each cycle (■varta) contains angas.

Anga

A section of a tala. Three types: Laghu (variable, = jati value), Dhrutam (2 beats), Anudhrutam (1 beat). A clap (kriya) marks each anga.

Jati

The value of the laghu anga: Chatusra=4, Tisra=3, Khanda=5, Misra=7, Sankirna=9.

Adi T■la

Chatusra Jati Triputa = 4+2+2 = 8 beats. The most common tala in Carnatic concerts.

Pallavi (■■■■■■)

The main refrain of a composition. Sets the tonal identity. Sung first and returned to throughout.

Anupallavi (■■■■■■■■■)

The second section, answering the pallavi, typically in the upper register.

Charanam (■■■■)

The verse or stanza body, elaborating the devotional or philosophical content.

Sahityamu

The lyrics text of a composition, aligned one syllable per swara beat.

■lapana

Free-form improvisation in a raga without tala. The raga is explored note by note.

Ragamalika

A composition or performance using multiple ragas in sequence. v3 detects up to 3 ragas.

Audava

A raga using only 5 swaras (pentatonic). Bilahari and Mohanam are audava ragas.

Sampoorna

A raga using all 7 swaras. Dheerasankarabharanam and Mayamalavagowla are sampoorna.

MusicXML

Standard XML format for sheet music interchange. v3 generates MusicXML 3.1 Partwise, importable in MuseScore, Finale, Sibelius.

OpenSheetMusicDisplay

Browser JavaScript library that renders MusicXML as professional SVG notation. Used in the Score tab.

GoMaa Raga Vidya v3 · Built with Node.js · Pure JavaScript · Offline-first · Data source: melakartajanyaragalist.csv (7,599 ragas) · MusicXML rendering: OpenSheetMusicDisplay · Sheet music export: MuseScore / Patantara compatible